SLE-4: Final Viva & Architecture Decision Record (ADR)
Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UAM076
Name: Shreya Guruprasad Patil
Division: B
Date: 11/10/2026
GitHub / Portfolio Link:https://github.com/patilshreya3/my-first-ai-agent-

---

PART A: Architecture Decision Record (ADR)

Title: Use of Breadth-First Search (BFS) / Depth-First Search (DFS) for State Exploration
Status: Accepted
Date: 11/10/2026

1. Context
While building our AI search agent, we needed to find a smart way to explore different states and reach the goal without getting lost or stuck. The main problem was deciding how the program should visit and remember paths so it doesn't run out of memory or get trapped in endless loops. We had to figure out the best searching method to handle the problem smoothly. This led us to test different algorithms like Breadth-First Search and Depth-First Search to see which one works best for our project.

2. Decision
We ultimately decided to use **Breadth-First Search (BFS)** as our core algorithm to explore the state space. This choice ensures that the agent systematically checks all neighboring paths level-by-level, guaranteeing that the shortest and most optimal path to the goal is found.

3. Alternatives Considered
Alternative 1: [e.g., Depth-First Search (DFS) - explored deeper paths first but risked getting stuck in infinite loops without depth limits.
Alternative 2: [e.g., Breadth-First Search (BFS) - guaranteed shortest path level-by-level but consumed more memory for large queues.

4. Rationale / Justification

We chose Breadth-First Search (BFS) because our SLE-2 test results showed it runs very fast and keeps memory usage low. Unlike DFS, which can get stuck or take the wrong path, BFS checks every step carefully to guarantee the shortest path, as seen in our performance graphs (`bfs-profile.svg`). Also, our SLE-3 design separated the program into clean parts (input, search, output), which helped the BFS queue work smoothly without crashing. Because of these strong testing numbers and clean design, BFS was clearly the best choice.

5. Consequences

Positive:

- Guaranteed to find the shortest and most accurate path to the goal because it checks every neighboring path step-by-step.
- Produced clear and reliable testing numbers during our SLE-2 performance profiling (bfs-profile.svg).

Negative / Trade-offs:

- Uses more memory space when the search graph becomes very wide, because it has to keep many nodes in the queue at the same time.

PART B: Short Summary of My SLE Journey

My SLE journey started with SLE-1, where I built the core search agent and python code for path exploration. In SLE-2, I tested and profiled the code to gather exact performance numbers and graphs. Next, in SLE-3, I designed the system structure by drawing clear C4 architecture diagrams. Finally, for SLE-4, I brought all this evidence together to write this Architecture Decision Record (ADR) and justify why Breadth-First Search was the best choice.

PART C: AI Contribution Note

AI tools used : Gemini
What AI helped with: Helped structure the ADR format and explain the guidelines clearly.
What I did myself: Wrote the code, performed the profiling tests, created the C4 diagrams, and selected the final technical decisions

.

PART D: Viva Preparation Notes

Why did you choose this algorithm?

We chose Breadth-First Search (BFS) because it systematically checks paths level-by-level, guaranteeing that we always find the shortest and most optimal path to the goal.

What alternatives did you reject?

We considered Depth-First Search (DFS), but we rejected it because it explores paths too deeply, risks getting stuck in infinite loops, and does not guarantee the shortest path.

What did your SLE-2 profiling show?

Our SLE-2 performance profiles showed that BFS runs efficiently with low execution time and manageable memory usage, as seen in our generated profiling graphs (bfs-profile.svg).

Explain your C4 architecture design:

Our SLE-3 C4 architecture separated the system into clear input, search, and output modules, which allowed the BFS queue data structure to pass states smoothly without crashing.

What did AI help with?

AI helped format the ADR document and explain the guidelines, while I wrote the actual code, performed the profiling tests, and created the diagrams myself.